

Intro to big data. Assignment 2

Methodology section

Firstly, SSH is enabled to allow remote interaction, and a Python virtual environment is configured to keep the project dependencies clean and isolated. This environment is then packaged, making it easy to redeploy the same setup elsewhere. To prepare the data for indexing, documents are pulled from a Parquet file and processed using Apache Spark. A subset of the data is selected for performance reasons, and each document is written as a text file. The data is also saved in CSV format.

The indexing workflow is managed by `index.sh`, which sets up everything needed for a Hadoop MapReduce job. It activates the virtual environment, bundles the Cassandra driver, and runs a script (`app.py`) to prepare the database. During this step, a keyspace is created in Cassandra along with four tables:

- `Inverted_index`. Stores terms along with document IDs and their frequencies for quick retrieval
- `Documents`. Holds metadata
- `Stats`. Maintains global metrics
- `Vocabulary`. Keeps track of term-specific stats

For indexing, the mapper script processes each document by making text preprocessing (tokenization, stop-words removing), outputting relevant term-document data along with metadata. After that, the reducer combines this data to calculate global stats and IDF scores before writing everything into Cassandra.

Finally, the `search.sh` script ensures the Cassandra driver is properly set up and then launches a Spark job. The BM25 ranking algorithm is used to evaluate how relevant documents are to a given query. It pulls everything it needs from Cassandra — term frequencies, document lengths, and vocabulary

stats — and ranks the results accordingly. The top matches are then displayed with their scores and titles.

Demonstration section

How to run?

```
docker compose up -d
```

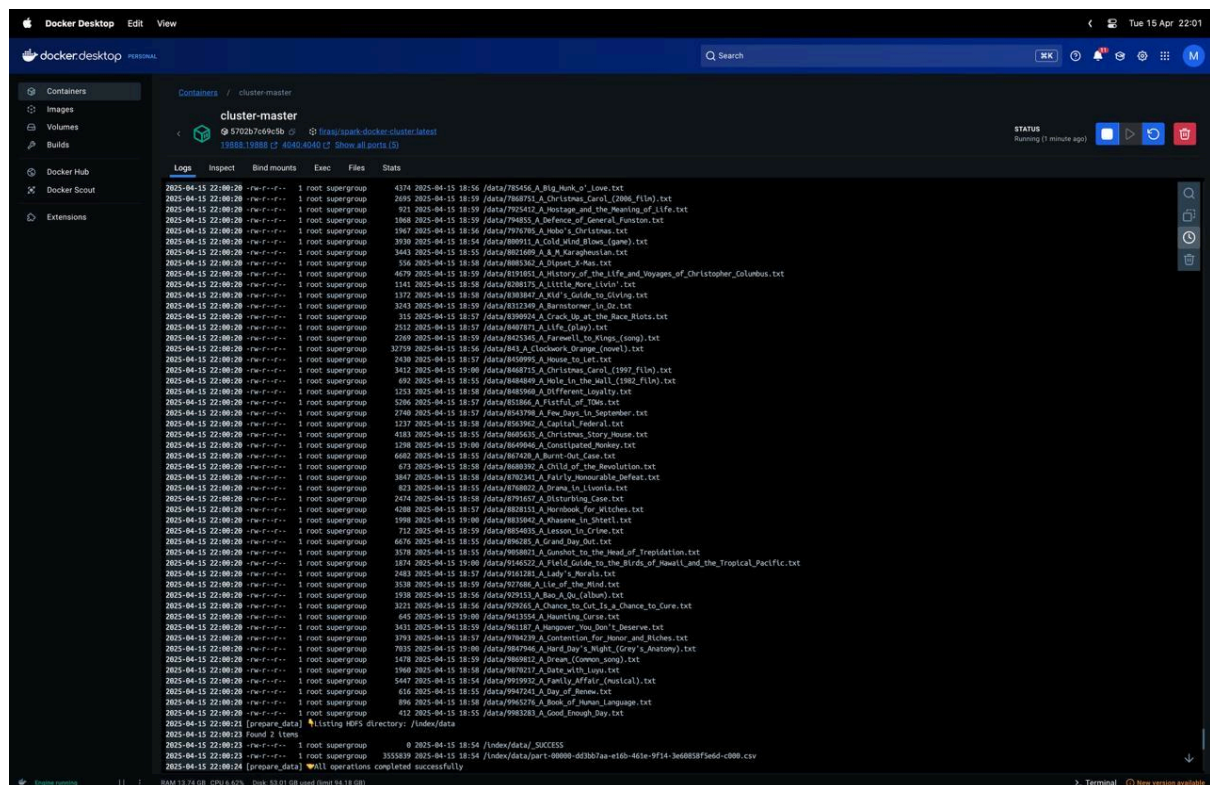


Figure 1: Documents are indexed

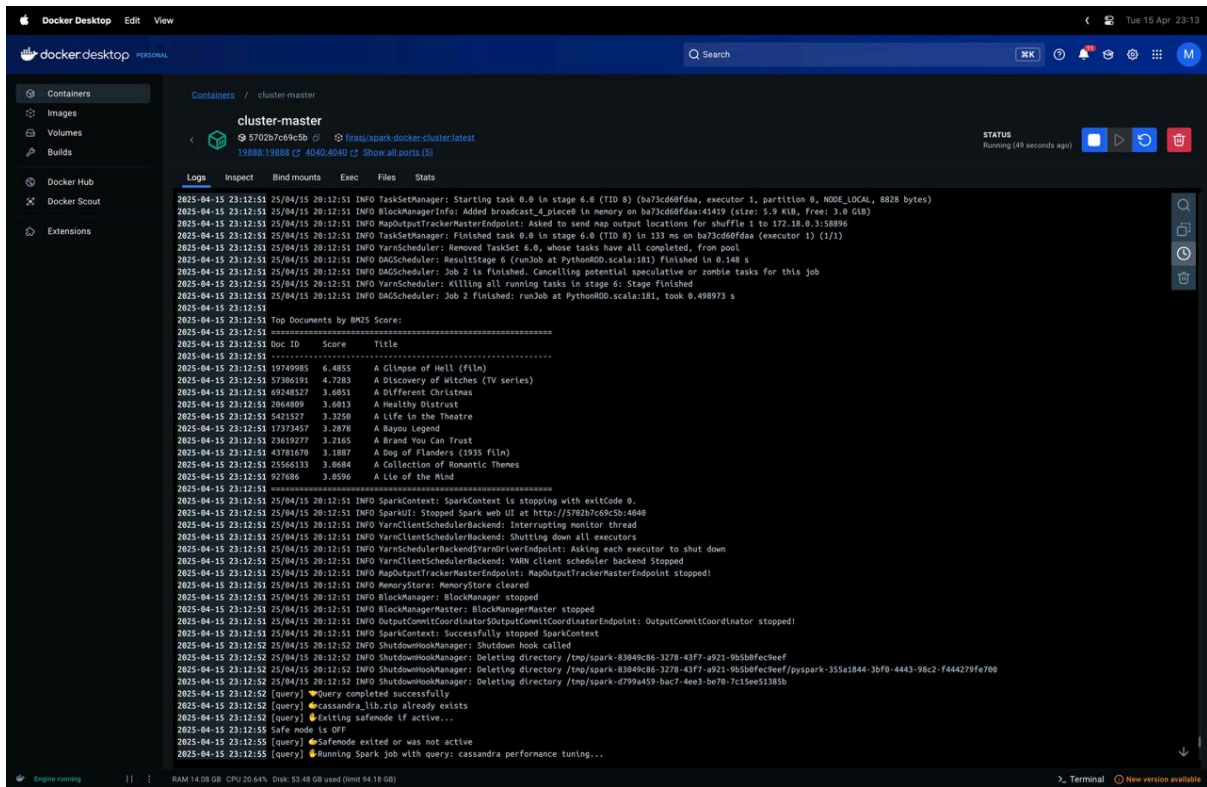


Figure 2. Query: "neural networks in production"

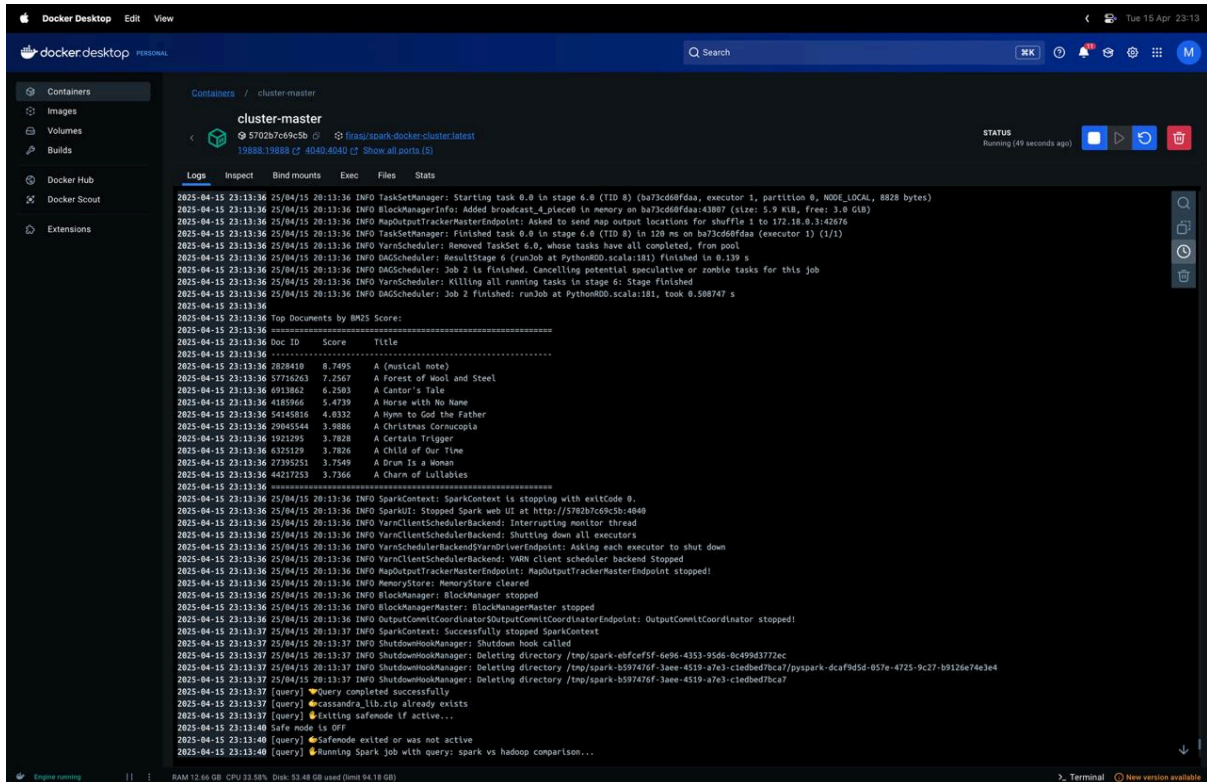


Figure 3. Query: "cassandra performance tuning"

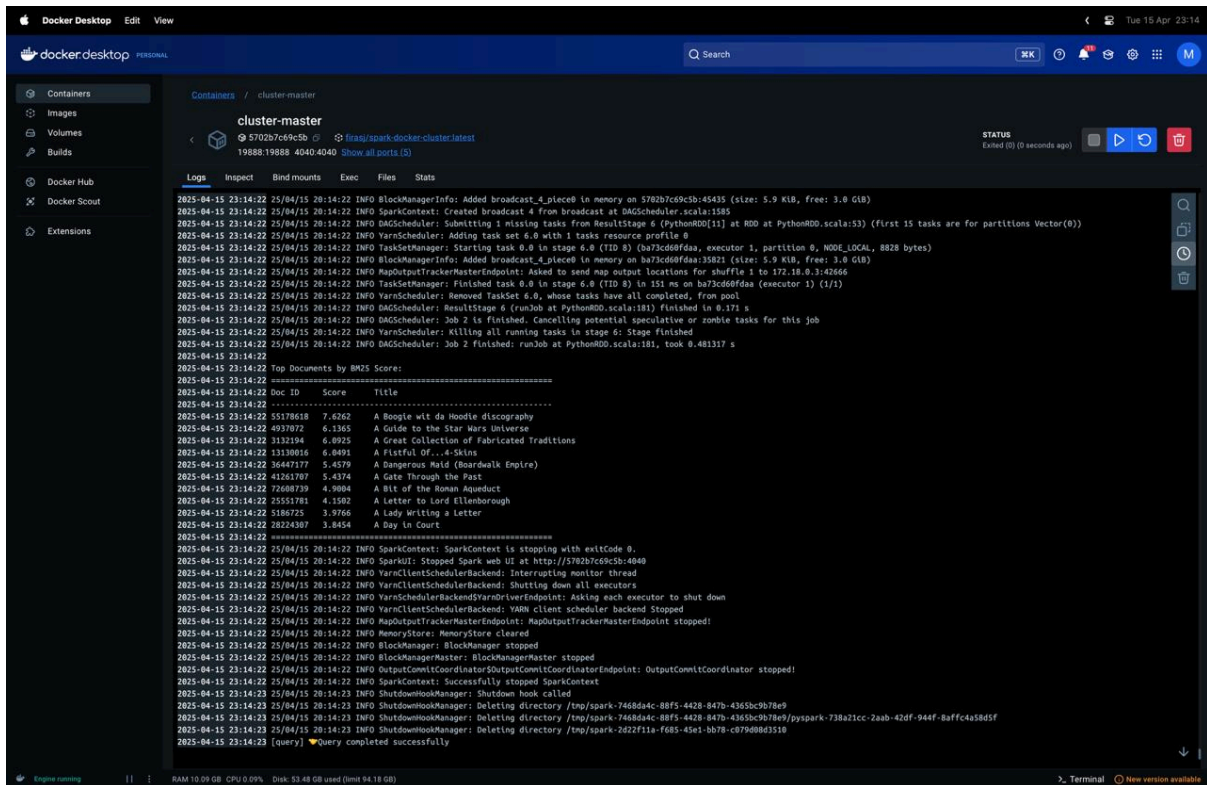


Figure 4. Query: "spark vs hadoop comparison"

For all queries, the results show that the system is working technically: it's processing the query, retrieving documents, and ranking them based on BM25. It's important to note that the results display only the titles of the documents—not their full content—so even if some titles don't seem directly related to the query, the actual relevance is based on the presence of keywords within the full text. Overall, the system performs well and returns accurate results based on the indexed content.